

* Always Remember: → Try Your Best to come up with the Recursive logic only, Because after that DP is just 5 minutes of work. — vHixem

* Problem: —

You are given an integer array `cost` where `cost[i]` is the cost of i^{th} step on a staircase. Once you pay the cost, you can either climb one or two steps.

You can either start from the step with index 0, or the step with index 1.

Return the minimum cost to reach the top of the floor.

idx → 0, 1, 2, (3) → So (idx = n) is Top
Input: `cost = [10, 15, 20]`
Output: 15
 $n = \text{array-size}$

You are given an integer array cost where cost[i] is the cost of i^{th} step on a staircase.

Once you pay the cost, you can either climb one or two steps.

You can either start from the step with index 0, or the step with index 1.

Return the minimum cost to reach the top of the floor.

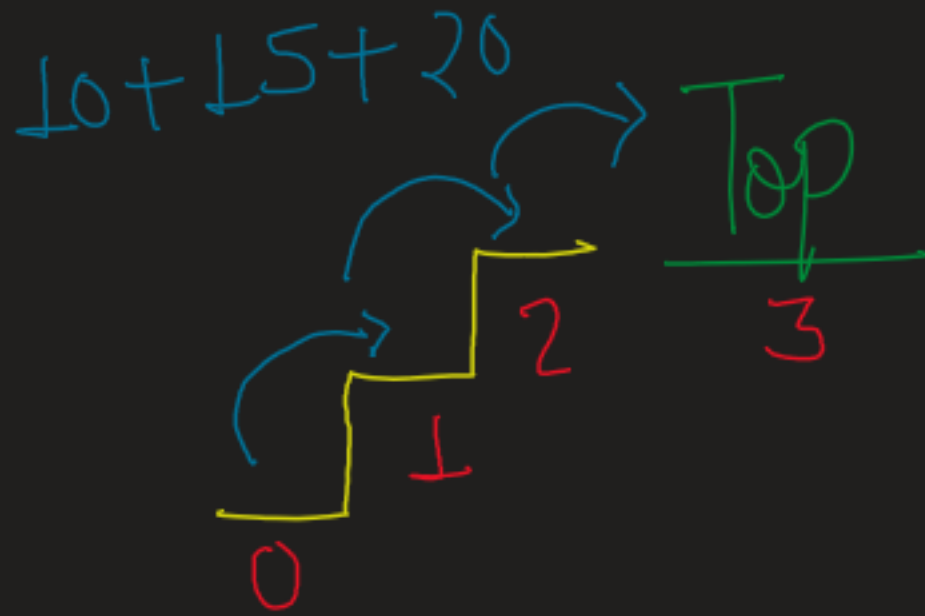
idx $\rightarrow$ 0, 1, 2, (3) $\rightarrow$ so (idx = n) is Top

Input: cost = [10, 15, 20]

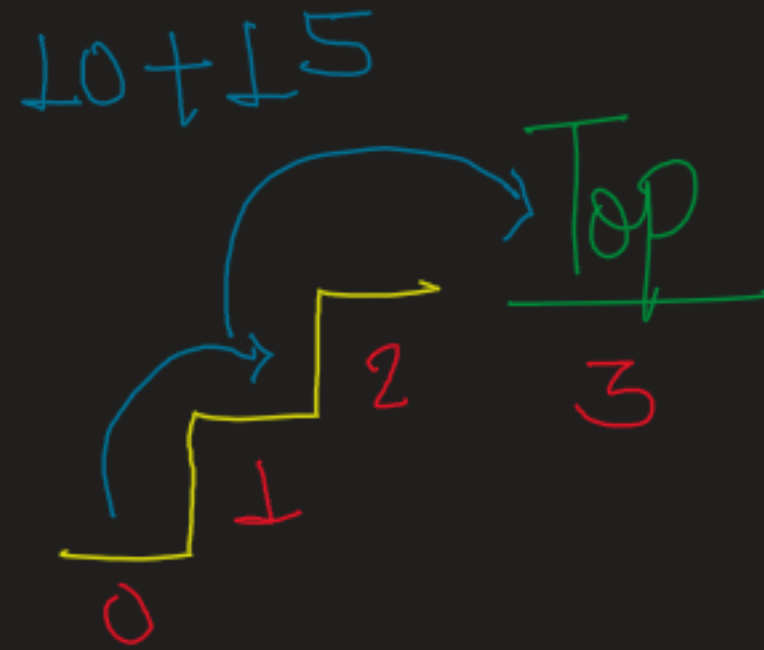
Output: 15

n = array-size

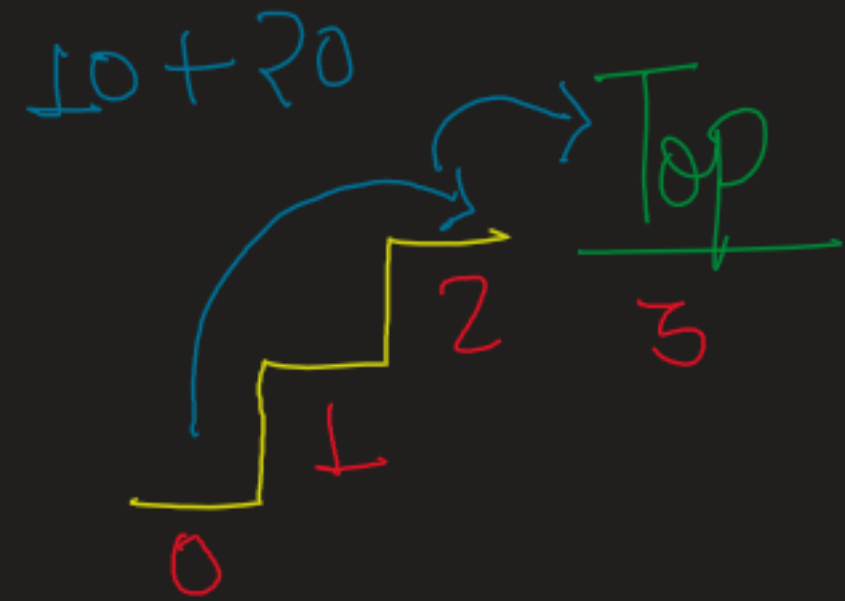
* Let start step = 0 *



cost = 45

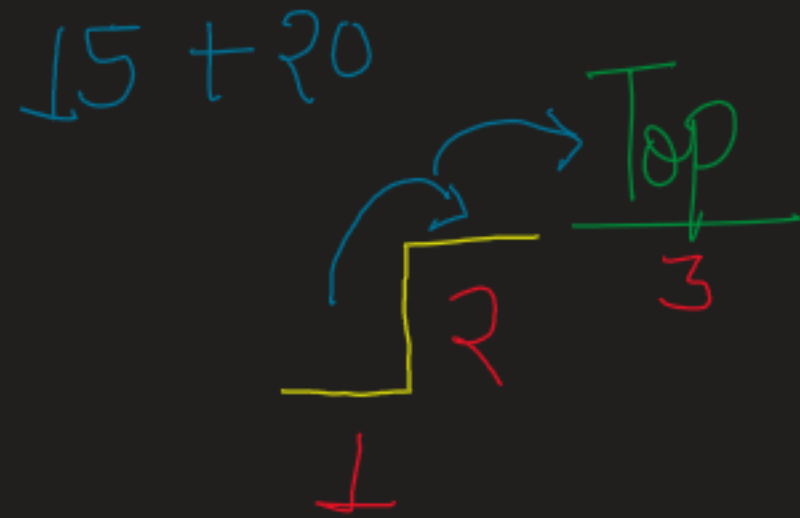


cost = 25



cost = 30

* Let start step = 1 *



cost = 35



cost = 15

- Take Minimum cost From starting from 0 → 25
- Take Minimum cost From starting from 1 → 15

→ Answer : $\min(25, 15) = 15$

* Build Understanding :-

idx $\rightarrow$ 0, 1, 2, (3) $\rightarrow$ So (idx = n) is Top
Input: cost = [10, 15, 20] $n = \text{array-size}$
Output If start is 0 index: 25

~ Suppose on $\text{idx} = 0$

climb1step :

$$\begin{array}{c} \text{cost}[0] + \text{cost}[1] \\ (10) \quad (15) \end{array} \Rightarrow 25$$

climb2step :

$$\begin{array}{c} \text{cost}[0] + \text{cost}[2] \\ (10) \quad (20) \end{array} \Rightarrow 30$$

final answer
 $\min(25, 30) = 25$

climb1step $\rightarrow$ $\text{cost}[i] + \text{result from } (i+1) \text{ index}$

climb2step $\rightarrow$ $\text{cost}[i] + \text{result from } (i+2) \text{ index}$

so find this recursively

climb1step $\rightarrow$ cost[i] + result from (i+1) index

climb2step $\rightarrow$ cost[i] + result from (i+2) index

$\downarrow$
so find this recursively

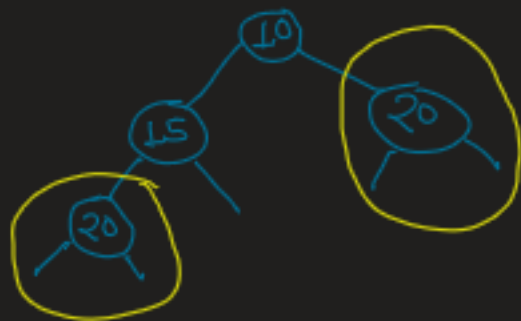
$\rightarrow$ overall, we want minimum result
so return $\min(\text{climb1step}, \text{climb2step})$

$\rightarrow$ if goes (i = n) in recursion then we are on Top
so if reached Top then what cost required
no cost, means 0, because we reached our destination.

* Time $\rightarrow O(2^N)$

$\rightarrow$ For any index we're exploring two options / possibilities.

Why It's taking Exponential time?



$\rightarrow$ Overlapping Sub problems (Repetitive work)

$\rightarrow$ Means when n is increased, the amount of sub-problem increases

$\rightarrow$ Solving this repetitive problems again and again is increasing time.

$\rightarrow$ This concludes that we could optimize time in our recursion.

$\rightarrow$ Now this hints to DP.

$\rightarrow$ So for recursion, we apply Top-Down DP,
So let's do Memorization.

$\rightarrow$ Now memorize it (Apply DP)

$\rightarrow$ let's code it